

Turning Chaos into Victory: Optimizing Random Team Composition in E-Sports Matches

Alex Holtzapple

Western Governors University School of Technology

D214: Data Analytics Graduate Capstone

February 19, 2025

Contents

Introduction	4
Research Question	7
Context	7
Research Question and Justification	7
Hypothesis.....	8
Data Collection	8
Data Extraction and Preparation	12
Formatting Match Data	12
Merging Champion Data	14
Aggregating Team Data	14
Analysis.....	16
Exploratory Analysis	16
Data Structure	16
Game Duration Outliers	16
Exploring Champion Roles.....	17
Logistic Regression Model.....	23
Base Model.....	23
Variance Inflation Factor.....	24
Recursive Feature Elimination	26
Model Interpretation.....	28
Random Forest Model	29
Baseline Model	29
Feature Importance	30
Final Model.....	31
Model Interpretation.....	32
Data Summary and Implications.....	34
Model Comparison.....	34
Predictive Performance	34

Interpretability	35
Practical Application	35
Analysis Limitations	36
Recommended Actions	37
Applying Model Rules	37
ARAM Team Building Tool	37
Future Research Direction	38
Incorporating Player Skill and In-Game Factors	38
Explore Other Machine Learning Techniques	38

Introduction

The competitive gaming industry has achieved massive growth over recent years, reaching a revenue of \$2.8 billion in 2024. With expected revenue to grow to over \$16 billion in 2033, competitive gaming has certainly become its own category of sport (Clement, 2025). Dubbed “eSports,” these competitive games offer exciting entertainment, and a wide variety of interesting data analytics opportunities. One of the most popular competitive games is League of Legends (LoL), an online game classified as a multiplayer online battle arena (MOBA). LoL boasts an average monthly player count of 131 million throughout 2024 (Gill, 2025).

LoL is a team-based game where two teams of five players compete to destroy the opposing team’s base, known as the Nexus. Players control unique characters called champions, each with distinct abilities that contribute to different team roles such as damage-dealing, defense, or healing/support. There are 2 main game modes available to play. One of which is on Summoner’s Rift, the game’s primary map, which is divided into three lanes and a jungle in the center (Riot Games, 2024). The other game mode is called All Random All Mid (ARAM), and takes place on Howling Abyss, a much smaller map consisting of a single-lane bridge connecting both teams’ bases (ARAM, 2025). The 2 modes lead to differences in how players can strategize champion selections.



Figure 1: Summoner's Rift map, the primary game mode for LoL (ARAM, 2025).



Figure 2: Howling Abyss map, the arena for ARAM game mode matches (ARAM, 2025).

Prior to starting a match on Summoner's Rift, teams select champions in a pre-match draft phase. Here, teams focus on gaining an advantage over the enemy team by choosing strong champions for each player's assigned role but also countering opponent champion choices. Summoner's Rift matches are well studied and mostly standardized consisting of the same champion roles for each match. Players choose specific roles such as top/tank, jungle, mid, support, and marksman/ad-carry, with each also being assigned a specific area of the map to start the game at (Riot Games, 2024a).

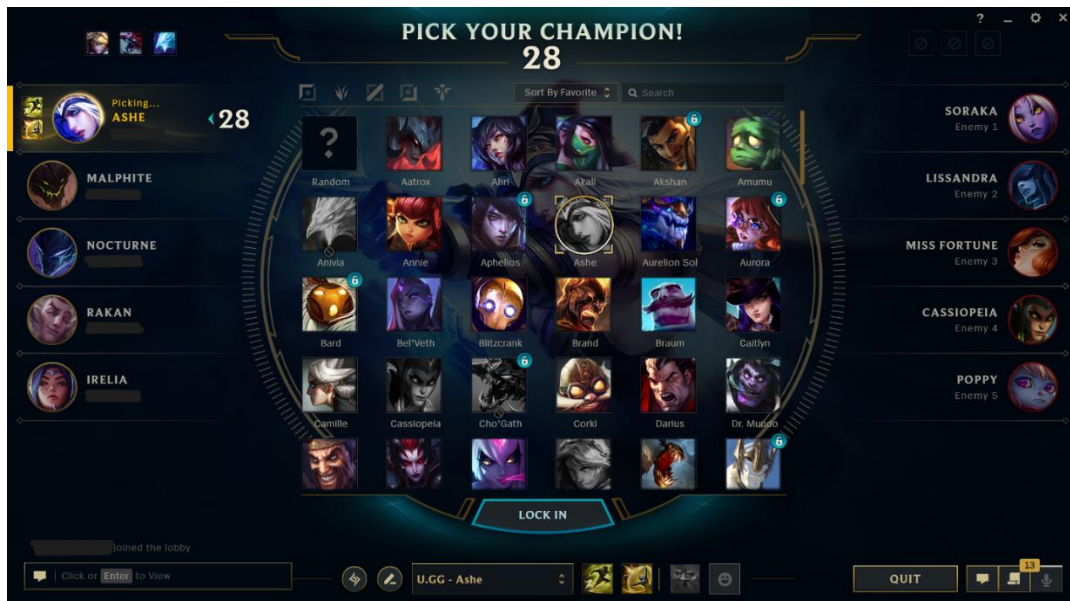


Figure 3: Summoner's Rift champion select screen showing enemy team champions.

In contrast, the ARAM game mode removes many strategic choices available in Summoner's Rift. Instead of selecting champions, players are assigned champions at random, and teams cannot see the enemy's champions until the match begins. However, players do have a few ways to alter their assigned champion (ARAM, 2025):

- Each player can re-roll up to two times to receive a different champion.
- Re-rolled champions are placed on a bench and may be picked by other teammates.
- Players can trade champions with each other before the match starts.

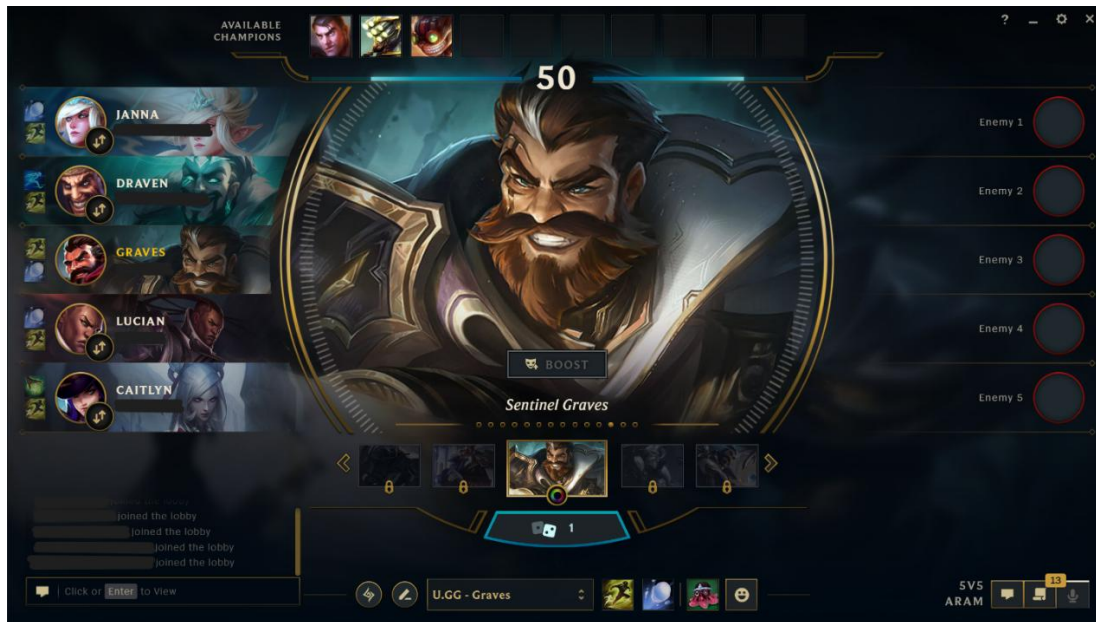


Figure 4: ARAM champion select screen with player champion pre-assigned and enemy champions hidden.

Unlike matches on Summoner's Rift, there are no side lanes or jungle areas on Howling Abyss, meaning that teams clash continuously in a central battlefield. Because of the constant engagement and limited strategic preparation, ARAM matches favor adaptability over predefined strategies.

The forced randomness of ARAM makes it a compelling challenge for predictive modeling and analytics. With 170 champions at the time of this study, there are quadrillions of variations of matchups that could result from the random assignment. Since team compositions are randomized, match outcomes should theoretically follow a 50/50 win rate over the long term. However, experienced players intuitively recognize that certain champions and compositions perform better in ARAM. Players often favor specific types, such as tanky frontliners or long-range marksmen, or actively avoid compositions that lack durability or crowd control. If these perceived advantages consistently impact match outcomes, it suggests that underlying team composition patterns exist within ARAM's chaotic champion selection process. Identifying and quantifying these patterns through predictive modeling could provide valuable insights into the factors that contribute to team success, despite the randomness of champion selection.

Research Question

Context

The ARAM game mode introduces a level of unpredictability not present in the game's primary mode, Summoner's Rift. Unlike traditional matches where teams strategically select their champions, ARAM assigns champions at random, theoretically ensuring an equal playing field. Given this randomness, teams should see around a 50% win-rate over many matches. However, player intuition and experience suggest that some team compositions lead to consistently higher win rates despite the lack of deliberate team selection.

This study seeks to determine whether data-driven predictive models can identify patterns in ARAM team compositions that can predict match outcomes with an accuracy greater than 50%, outperforming random guessing. If such a model is possible, it would indicate that despite ARAM's random nature, patterns exist in team composition that make certain teams more likely to win. In practical terms, this would suggest that players could apply insights from the models, such as when to use a re-roll or switch with a benched champion, to increase their win rates beyond the 50% expected from pure randomness.

Research Question and Justification

The core research question guiding this study is:

“Can a predictive model determine the winning team in an ARAM match based on team composition with an accuracy greater than random guessing (50%)?”

This research question is significant because if certain team compositions consistently achieve higher win rates, it would suggest that ARAM is not entirely dictated by randomness and that inherent composition-based advantages exist within the mode. This study aims to quantify what many experienced players intuitively recognize—that some champion combinations provide a strategic edge despite the lack of pre-match planning.

ARAM is a widely played format, and tournaments are occasionally held for it (*LoL Tournaments*, 2023). Insights from a successful predictive model could be valuable for frequent ARAM players, helping them make more informed re-roll decisions, adapt strategies based on team composition, and improve their chances of success despite the game mode's randomness.

Hypothesis

To explore this research question, the study will test the following hypotheses:

Null Hypothesis (H_0): *A predictive model cannot predict match outcomes better than random guessing ($\leq 50\%$ accuracy).*

Alternative Hypothesis (H_1): *A predictive model can predict match outcomes better than random guessing ($> 50\%$ accuracy).*

Since ARAM assigns champions randomly, match outcomes should theoretically resemble a coin flip over the long term, meaning a model should not be able to predict the winner with greater than 50% accuracy unless composition-based advantages create measurable patterns that can be exploited. This forms the basis of the null hypothesis, which assumes that while composition may play a role in gameplay, its effects are either too small, inconsistent, or unpredictable for a model to capture effectively. In this case, random champion assignments would be just as effective as any model-based prediction.

However, many players intuitively believe that certain combinations consistently lead to better results, suggesting that players may be able to strategically “beat” the randomness by making informed re-roll decisions to optimize their team composition. If the alternative hypothesis is supported, meaning a predictive model can achieve greater than 50% accuracy, this would indicate that team composition alone provides enough predictive power to improve match outcome predictions beyond random chance.

A result in favor of the alternative hypothesis would demonstrate that despite the randomness of champion selection, ARAM match outcomes are not purely dictated by chance. This aligns with broader trends in esports analytics, where machine learning models have been used to uncover hidden patterns in player behavior, team strategies, and competitive balance. If successful, this research could provide actionable insights for ARAM players, helping them make better decisions and adjust their strategies based on their assigned team composition, ultimately improving their win rates over the long term.

Data Collection

The dataset for this study was collected using the game API, which provides access to detailed match data for LoL, including player statistics, match outcomes, and champion information (Riot Games, n.d.).

A set of functions were created using Python to make requests to the API and retrieve the necessary data. The collection process consisted of the following steps:

1. **Collect Champion Data** – A function was written to query high level champion data, such as name, id, and base stats, and save the result out to a file. This data is later merged with match data to aggregate champion statistics on a match and team level. Figure 5 shows the function used:

```
def retrieve_champ_info():
    champs_r = requests.get('http://ddragon.leagueoflegends.com/cdn/15.2.1/data/en_US/champion.json')
    champs = json.loads(champs_r.text)['data']
    champion_data = []
    for champ in champs.values():
        roles = champ['tags']
        if len(roles) > 1:
            primary_role, secondary_role = roles[0], roles[1]
        else:
            primary_role, secondary_role = roles[0], None
        champ_info = {
            'name': champ['name'],
            'id': champ['id'],
            'key': champ['key'],
            'primary_role': primary_role,
            'secondary_role': secondary_role
        }
        champ_info.update(champ['info'])
        champ_info.update(champ['stats'])
        champion_data.append(champ_info)
    champions_df = pd.DataFrame(champion_data)
    return champions_df

champ_info = retrieve_champ_info()
champ_info.to_csv('champ_info.csv', index = False)
```

Figure 5: Custom function to retrieve, parse, and store champion statistics.

The champion statistics retrieved consisted of the following metrics:

- *Name* – Champion's display name.
 - *Id* – Champion's back-end ID name.
 - *Key* – Champion's numeric ID.
 - *Primary_role* – Champion's assigned primary role.
 - *Secondary_role* – Champion's assigned secondary role, if applicable
 - *Attack, Defense, Magic, Difficulty* – Champion's assigned rating for each category on a 1 to 10 scale.
 - *HP, HPperlevel, MP, MPperlevel ...* – detailed statistics at champion's base level in game, not used in the final analysis.
2. **Retrieve Player Matches** – Another function queries the API to retrieve match IDs for a single player. This function applies a few filters including a cutoff date to collect matches played only after a recent game update, and a game mode filter to ensure only ARAM matches are collected.

To avoid any bias by collecting too many matches from one player, a sample size limit of 20 matches per player was also set. The data retrieved from this

function consists only of a match id column and is used as an input into another function. Figure 6 demonstrates the match collection function:

```
def retrieve_matches(apikey, puuid, sample=20, starttime=''):
    if starttime != '':
        starttime = 'startTime={}&'.format(starttime)
    matchlist = []
    matches = requests.get('https://americas.api.riotgames.com/lol/match/v5/matches/by-puuid/{}/ids?{}queue=450&count={}&api_key={}'.format(puuid, starttime, sample, apikey))
    while matches.status_code == 429:
        print('Hit call limit, waiting 30s')
        time.sleep(30)
    matches = requests.get('https://americas.api.riotgames.com/lol/match/v5/matches/by-puuid/{}/ids?{}queue=450&count={}&api_key={}'.format(puuid, starttime, sample, apikey))
    if matches.status_code != 200:
        ValueError('Received error: ' + str(matches.status_code))
    matchlist.extend(json.loads(matches.content))
    print('{} matches retrieved'.format(len(matchlist)))
    return matchlist
```

Figure 6: Custom function to retrieve a list of matches for a single player.

3. **Retrieve Detail Match Data** – The match IDs retrieved in step 2 are then used as inputs into a function that retrieves the details of the match.

```
def retrieve_single_match(apikey, matchid):
    indiv_match_r = requests.get('https://americas.api.riotgames.com/lol/match/v5/matches/{}?api_key={}'.format(matchid, apikey))
    while indiv_match_r.status_code == 429:
        print('Hit call limit, waiting 30s')
        time.sleep(30)
    indiv_match_r = requests.get('https://americas.api.riotgames.com/lol/match/v5/matches/{}?api_key={}'.format(matchid, apikey))
    if indiv_match_r.status_code == 200:
        match_json = json.loads(indiv_match_r.text)
    else: raise ValueError('Invalid response code {}'.format(str(indiv_match_r.status_code)))
    return match_json
```

Figure 7: Custom function to retrieve details of a single match.

4. **Parse Detail Match Data** – Lastly, the detail match data must be parsed into a more compact table format. The original data consists of a JSON response with many layers providing an immense level of detail of the match (Riot Games, n.d.). Most of this data is not necessary for this analysis though and only the relevant features such as players, duration, champions selected, and match result are kept.

Additionally, several filters are used to ensure the match was fully completed, is of the ARAM game mode, and meets a minimum timeframe of 8 minutes. The 8-minute limit is implemented to remove any matches that were forfeited early due to a missing player or technical glitch.

Players who left the game early are checked for again by comparing each team's player levels. Because of the close-quarters nature of ARAM, players should mostly level up together. Any matches that had a player more than 3 levels behind the rest of their team could be considered as having left the game early and are filtered out of the dataset. Figure 8 shows the match data parsing function.

The final data collected for each match consisted of:

- *Match ID* – Unique identifier string for each match.
- *Team1 Result* – Boolean indicating team 1 victory or defeat.
- *Team2 Result* – Boolean indicating team 2 victory or defeat.
- *Game Duration* – Length of match in seconds.

- *Puuid* – Unique player ID for each player in the match.
- *ChampionID* – Numeric champion ID for each player.
- *ChampionName* – Champion back-end name for each player.

```
def parse_match_data(match_data):
    game_status = match_data["info"]["endOfGameResult"]
    game_mode = match_data["info"]["gameMode"]
    platform_id = match_data["info"]["platformId"]
    game_duration = match_data["info"]["gameDuration"]
    if game_status != "GameComplete" or game_mode != "ARAM" or platform_id != "NA1" or game_duration < 480:
        return None
    team_levels = {100: [], 200: []}
    player_data = []
    for participant in match_data["info"]["participants"]:
        champ_level = participant["champLevel"]
        team_id = participant["teamId"]
        player_info = {
            "puuid": participant["puuid"],
            "championId": participant["championId"],
            "championName": participant["championName"],
            "teamId": participant["teamId"]
        }
        player_data.append(player_info)
        team_levels[team_id].append(champ_level)
    for team_id, levels in team_levels.items():
        if levels:
            level_range = max(levels) - min(levels)
            if level_range > 3:
                return None
    team_results = {team["teamId"]: team["win"] for team in match_data["info"]["teams"]}
    return {
        "team1_result": team_results.get(100), # Team 1 (100)
        "team2_result": team_results.get(200), # Team 2 (200)
        "game_duration": game_duration,
        "players": player_data
    }
```

Figure 8: Custom function to apply filters and parse detail match data.

The functions for each step were set up in a loop to retrieve a total of 20,012 valid ARAM matches. With each match providing two data points (one for each team), a raw total of 40,024 data points were collected.

One advantage of this methodology is that the automation of API calls through custom Python functions allowed for a structured, scalable, and reproducible data collection process. Instead of manually querying individual matches, the functions handled match retrieval, filtering, and parsing automatically, reducing the likelihood of human error and ensuring consistency across all collected data. Using the game's own API also provided an advantage in that the data was sourced through official game records, ensuring accuracy and completeness compared to manually recorded or third-party data.

However, a key disadvantage of this approach is that the game API enforces strict rate limits on data requests (Riot Games, n.d.). As a result, the full data collection process took over 6 hours to complete. While this was manageable for the scope of this study, larger-scale analyses requiring hundreds of thousands of matches would face significant

time constraints, requiring either a longer collection window, or coordination with game developers to increase the request rate. One way to overcome this challenge was using built in checks and delays for each API function. If the API returned a rate limit code (error code 429), the function would delay for 30 seconds to allow the rate limit to expire, then continue with another batch of requests. Although this did not improve the total time needed for data collection, it did ensure the collection was completely automated and ran uninterrupted until the desired number of matches was collected.

Data Extraction and Preparation

Once the raw match data was collected, additional processing steps were applied to transform it into a structured dataset for analysis. This process included parsing match data into a Pandas DataFrame, merging champion information, and aggregating key statistics at the team level to create a final dataset for modeling.

Formatting Match Data

The match data retrieved from the game's API was returned in JSON format, containing nested structures for game details, team results, and player statistics. To make this data usable for analysis, a function was implemented to parse and restructure the JSON into a tabular format using pandas DataFrames.

```

In [11]: json_data['[REDACTED]']
Out[11]:
{'team1_result': True,
 'team2_result': False,
 'game_duration': 928,
 'players': [{'puuid': '[REDACTED]',
                'championId': 127,
                'championName': 'Lissandra'},
              {'puuid': '[REDACTED]',
                'championId': 82,
                'championName': 'Mordekaiser'},
              {'puuid': '[REDACTED]',
                'championId': 62,
                'championName': 'MonkeyKing'},
              {'puuid': '[REDACTED]',
                'championId': 104,
                'championName': 'Graves'},
              {'puuid': '[REDACTED]',
                'championId': 51,
                'championName': 'Caitlyn'},
              {'puuid': '[REDACTED]',
                'championId': 98,
                'championName': 'Shen'},
              {'puuid': '[REDACTED]',
                'championId': 112,
                'championName': 'Viktor'},
              {'puuid': '[REDACTED]',
                'championId': 517,
                'championName': 'Sylas'},
              {'puuid': '[REDACTED]',
                'championId': 54,
                'championName': 'Malphite'},
              {'puuid': '[REDACTED]',
                'championId': 142,
                'championName': 'Zoe'}]}

```

Figure 9: Single match data in original JSON format.

Each match entry was first broken down into individual players, with match level results like game duration, winning team, and match ID repeated. This set-up allowed for easier merging of champion statistics in a later step. Figure 10 shows the resulting DataFrame structure.

```

      match_id  team  champion_id  champion_name  game_duration  win
0  [REDACTED]    1         61      Orianna         1294    True
1  [REDACTED]    1        238        Zed         1294    True
2  [REDACTED]    1         78      Poppy         1294    True
3  [REDACTED]    1         29      Twitch         1294    True
4  [REDACTED]    1         98        Shen         1294    True

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200120 entries, 0 to 200119
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   match_id        200120 non-null  object
1   team            200120 non-null  int64
2   champion_id     200120 non-null  int64
3   champion_name   200120 non-null  object
4   game_duration   200120 non-null  int64
5   win             200120 non-null  bool

```

Figure 10: Match data parsed to individual player entries for easier merging with champion data.

Merging Champion Data

Next, champion detail data was merged to the player data using the common *champion_id* column. The resulting DataFrame included the statistics for each player's champion.

```
RangeIndex: 177370 entries, 0 to 177369
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   match_id              177370 non-null object
1   team                  177370 non-null int64
2   champion_id           177370 non-null int64
3   champion_name         177370 non-null object
4   game_duration         177370 non-null int64
5   win                   177370 non-null bool
6   name                  177370 non-null object
7   id                    177370 non-null object
8   key                   177370 non-null int64
9   primary_role          177370 non-null object
10  secondary_role         136222 non-null object
11  attack                177370 non-null int64
12  defense               177370 non-null int64
13  magic                 177370 non-null int64
14  difficulty             177370 non-null int64
15  hp                    177370 non-null int64
16  hpperlevel            177370 non-null int64
17  mp                    177370 non-null int64
18  mpperlevel            177370 non-null float64
19  movespeed             177370 non-null int64
20  armor                 177370 non-null int64
21  armorperlevel         177370 non-null float64
22  spellblock            177370 non-null int64
23  spellblockperlevel    177370 non-null float64
24  attackrange           177370 non-null int64
25  hpregen               177370 non-null float64
26  hpregenperlevel       177370 non-null float64
27  mpregen               177370 non-null float64
28  mpregenperlevel       177370 non-null float64
29  attackdamage          177370 non-null int64
30  attackdamageperlevel  177370 non-null float64
31  attackspeedperlevel   177370 non-null float64
32  attackspeed           177370 non-null float64
```

Figure 11: Player level data merged with champion detail statistics.

Aggregating Team Data

The dataset at this point consisted of 33 columns. However, most of these are related to detailed base statistics for each champion. These statistics such as *hp* (health points), *movespeed*, *attackdamage*, etc. are drastically modified during a match by the items a player purchases (Item, 2025). Across champions, base statistics also don't vary significantly. For example, most champions start with a *movespeed* from 325 to 350, and *attackdamage* from mid 50s to upper 60s (List of champions/base statistics, 2024). Champions also aren't selected based on their base level performance but rather their overall role and synergies with other champions. Therefore, the detailed statistics columns are removed in favor of the more interpretable champion role assignments.

Overall ratings in the main aspects of *attack*, *defense*, *magic*, and *difficulty* are also retained. These ratings are not specific statistics but rather a general indicator of the champion's playstyle that is consistent throughout the match. These aspects are rated on a

scale of 1 to 10. The *primary_role* column is a categorical with 6 possible values. To structure the roles for analysis, the column is one-hot encoded, producing a new Boolean column for each champion role.

```
RangeIndex: 177370 entries, 0 to 177369
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   match_id                             177370 non-null object
1   team                                  177370 non-null int64
2   game_duration                         177370 non-null int64
3   win                                   177370 non-null bool
4   attack                               177370 non-null int64
5   defense                              177370 non-null int64
6   magic                                177370 non-null int64
7   difficulty                           177370 non-null int64
8   primary_role_Assassin                177370 non-null bool
9   primary_role_Fighter                 177370 non-null bool
10  primary_role_Mage                     177370 non-null bool
11  primary_role_Marksman                 177370 non-null bool
12  primary_role_Support                  177370 non-null bool
13  primary_role_Tank                     177370 non-null bool
```

Figure 12: Player level data after filtering champion base stats and one-hot encoding *primary_role*.

To be able to analyze the data at the team level, the ratings and roles were aggregated by match and team using the *match_id* and *team* indices. The champion ratings, *attack*, *defense*, *magic*, and *difficulty* were averaged for each team. Each of the primary roles was summed, producing a total count for each team. The column *game_duration* was aggregated for each match as an integer and the column, *win*, aggregated for each team as a Boolean value.

```
   match_id  team  ...  game_duration  win
0  [REDACTED]  1  ...      700.0  False
1  [REDACTED]  2  ...      700.0   True
2  [REDACTED]  1  ...     1219.0   True
3  [REDACTED]  2  ...     1219.0  False
4  [REDACTED]  1  ...     1760.0  False

[5 rows x 14 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35474 entries, 0 to 35473
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   match_id                             35474 non-null object
1   team                                  35474 non-null int64
2   attack_mean                          35474 non-null float64
3   defense_mean                         35474 non-null float64
4   magic_mean                           35474 non-null float64
5   difficulty_mean                      35474 non-null float64
6   primary_role_Assassin_sum            35474 non-null int64
7   primary_role_Fighter_sum             35474 non-null int64
8   primary_role_Mage_sum                35474 non-null int64
9   primary_role_Marksman_sum            35474 non-null int64
10  primary_role_Support_sum              35474 non-null int64
11  primary_role_Tank_sum                 35474 non-null int64
12  game_duration                         35474 non-null float64
13  win                                   35474 non-null bool
```

Figure 13: Data aggregated to the match - team level for final analysis.

Analysis

Exploratory Analysis

Before building predictive models, the dataset was examined to ensure data integrity, identify patterns, and detect potential issues such as outliers or multicollinearity that may affect the modeling process. To begin, the dataset was inspected for structure, data types, and missing values.

Data Structure

Figure 13 shows that each of the 14 columns has a total of 35,474 non-null entries, confirming that no values are missing. The data type for each column matches expectations with average statistics recorded as float types, role counts as integers, and the response variable *win* as a Boolean value. Although they will not be used as features in the model, *match_id*, *team*, and *game_duration* also have correct data types of object (string), integer (team 1 or team 2), and float.

Game Duration Outliers

ARAM matches typically fall within a certain time range and very short or very long match durations may be an indicator of abnormalities in the way the match was played, or in the team compositions. These outliers should be removed to ensure only valid matches are analyzed and they do not skew model results. Matches on the shorter end were already removed during the data collection process which set a hard filter of 480 seconds or 8 minutes, as matches ending prior to 8 minutes were likely an early forfeit due to a missing player or other technical glitch.

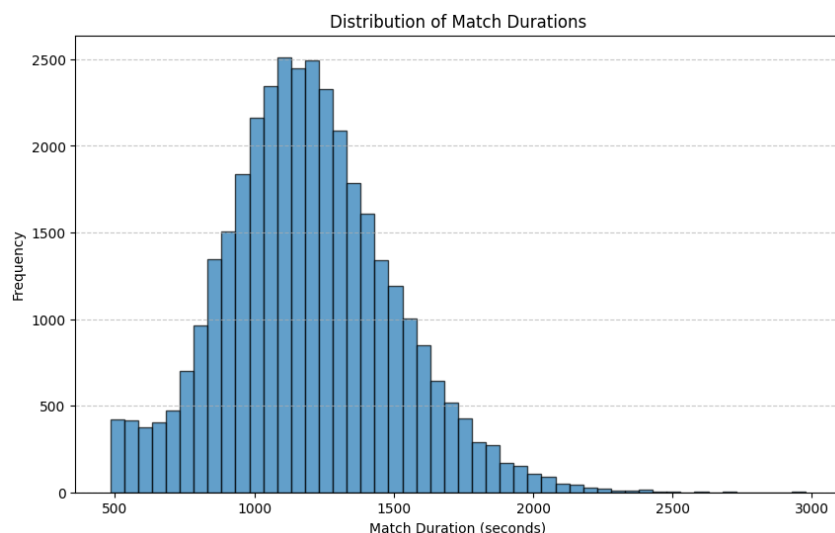


Figure 14: Distribution of match durations prior to removing high-end outliers.

Most matches fell within the 1000 to 1500 second range (16 to 25 minutes). There is a long tail to the distribution beyond 2000 seconds, with a few matches even reaching 3000 seconds (50 minutes). The inter-quartile range (IQR) method was employed to remove these high-end outliers. The IQR method consists of calculating an upper bound equal to $1.5 \times$ the difference between the 1st and 3rd quartiles. Matches with a duration above the upper bound or below the lower bound are considered outliers and removed from the dataset (Chaudhary, 2024). The lower bound was 408.5 seconds, meaning no additional matches were removed. The upper bound was 1964.5 seconds. A total of 216 matches were above this duration and removed from the dataset. Since matches have an entry for each team, a total of 432 data points were removed. The remaining dataset consisted of 35,042 data points with a max duration of 1964 seconds, minimum of 483 seconds, and average of 1185.2 seconds. Figure 15 shows the updated distribution with outlier matches removed.

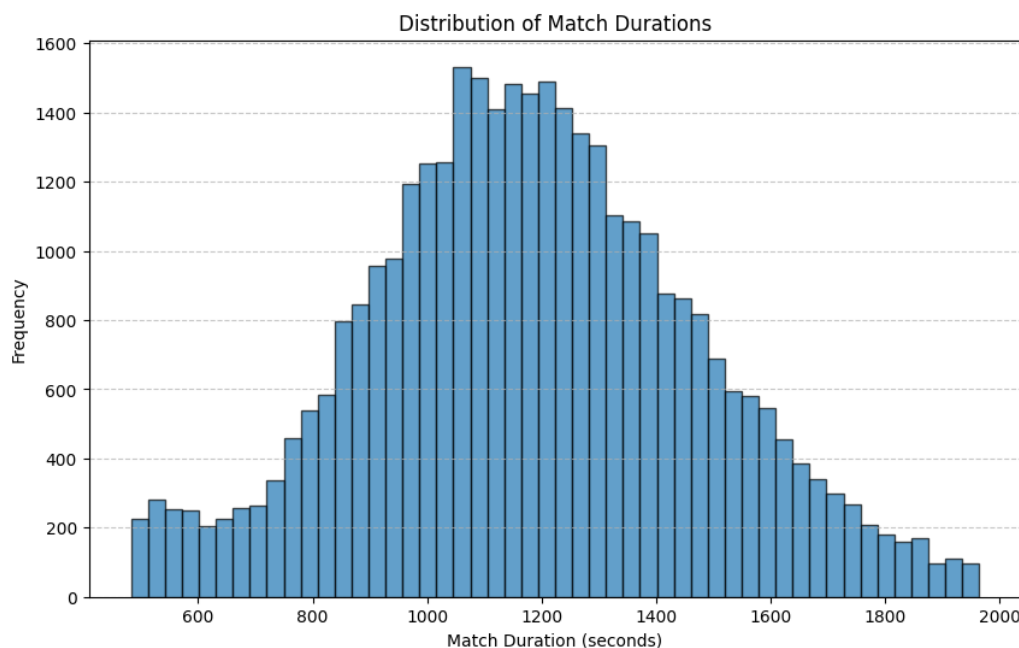


Figure 15: Match duration distribution with outlier matches removed.

Exploring Champion Roles

Champion roles are a key focus of this study and are important to understand how they are present in the data. Figure 16 shows the distribution of teams that had at least one champion of that role present. It's clear that Mage and Marksman champions are preferred among players, being represented at least once in 28,182 (80.4%) and 27,620 (78.8%) teams respectively. This contrasted with the Assassin and Support roles which are only present in 12,238 (34.9%) and 14,932 (42.6%) teams respectively. This is the first indication

in the data of players demonstrating intuition regarding successful champions in ARAM. If the selection was truly random, the data should show a more even split among all roles.

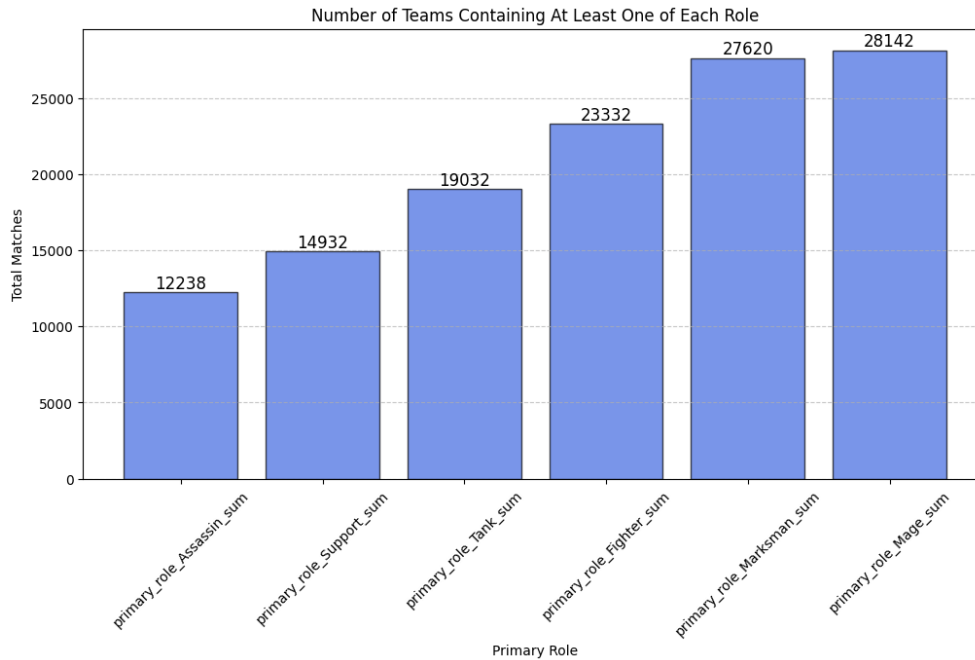


Figure 16: Teams containing at least 1 of the specified role, showing clear preference for Mage and Marksman champions.

Teams with an abnormal number of each role also need to be considered since they are unlikely to have a high enough sample size to be useful for modeling. For example, players are unlikely to play a match using the same champion role for all 5 players. Table 1 shows a number of teams with each possible number of champions per role, and confirmed that teams with 5, or even 4 of the same role were rare. Assassin and Support roles did not have any teams where all 5 champions were of that role. Teams with 4 of the same role again confirm that Mage and Marksman champions are preferred.

Role	0	1	2	3	4	5
Assassin	22804	10522	1589	121	6	0
Fighter	11710	15152	6755	1317	107	1
Mage	6900	14665	10315	2821	334	7
Marksman	7422	15876	9424	2128	187	5
Support	20110	12394	2369	163	6	0
Tank	16010	14991	3708	319	12	2

Table 1: Role count by teams show teams with uniform compositions are rare.

To avoid imbalances affecting the analysis, single role count within teams was capped at 3 champions of a single role. This had the advantage of preventing rare compositions from skewing results but resulted in lost data. It also limited interpretation to some extent since teams with 4 or 5 stacked roles were unseen by the models. Any

recommendations were limited to teams with compositions only up to 3 of the same role. A total of 667 teams were removed, leaving the dataset with a total of 34,375 data points.

Champion Role Win Rates

To assess an initial impact of team composition on match outcome, the win rates for each champion role were calculated based on their count within teams. Figures 17 through 22 show the win rate distribution for each role.

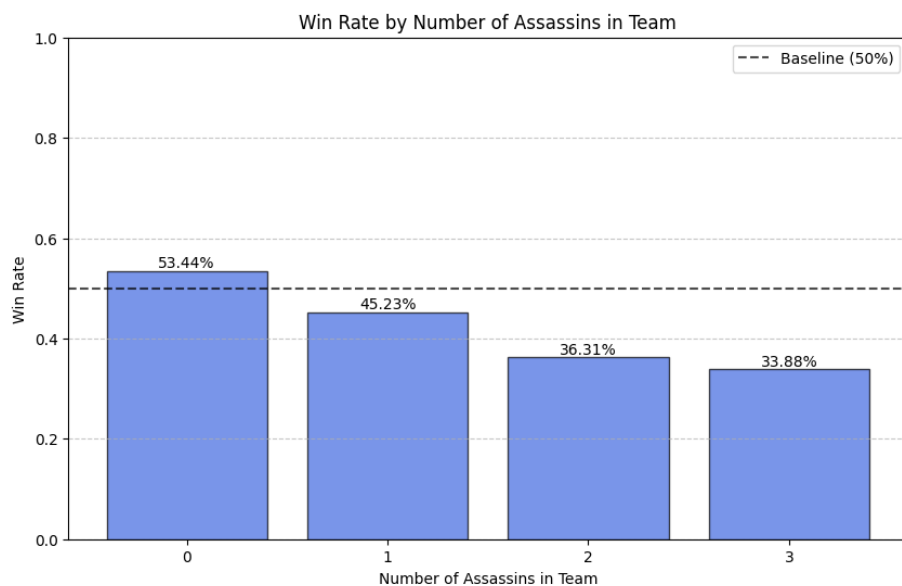


Figure 17: Win rate decreases as more Assassins are present.

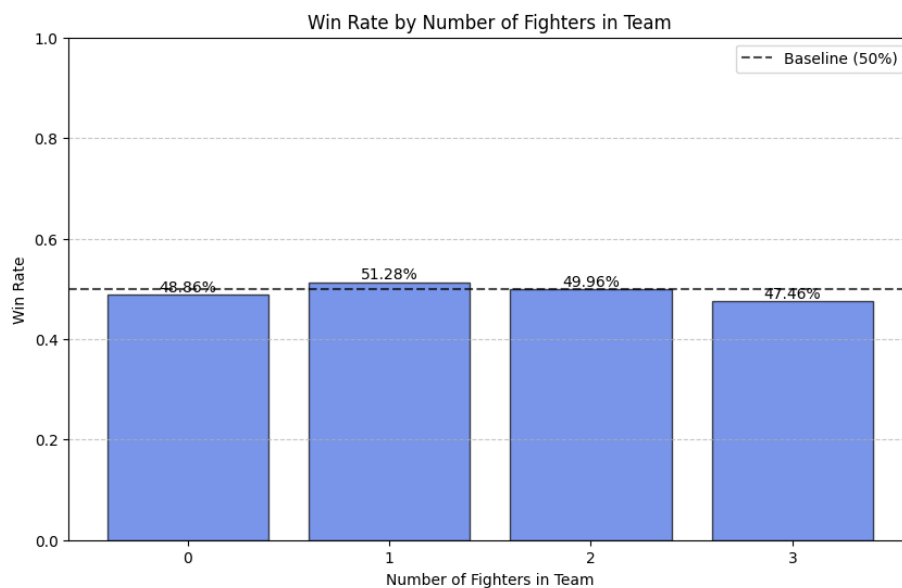


Figure 18: Win rate for Fighters peaks with having 1 Fighter champion on the team.

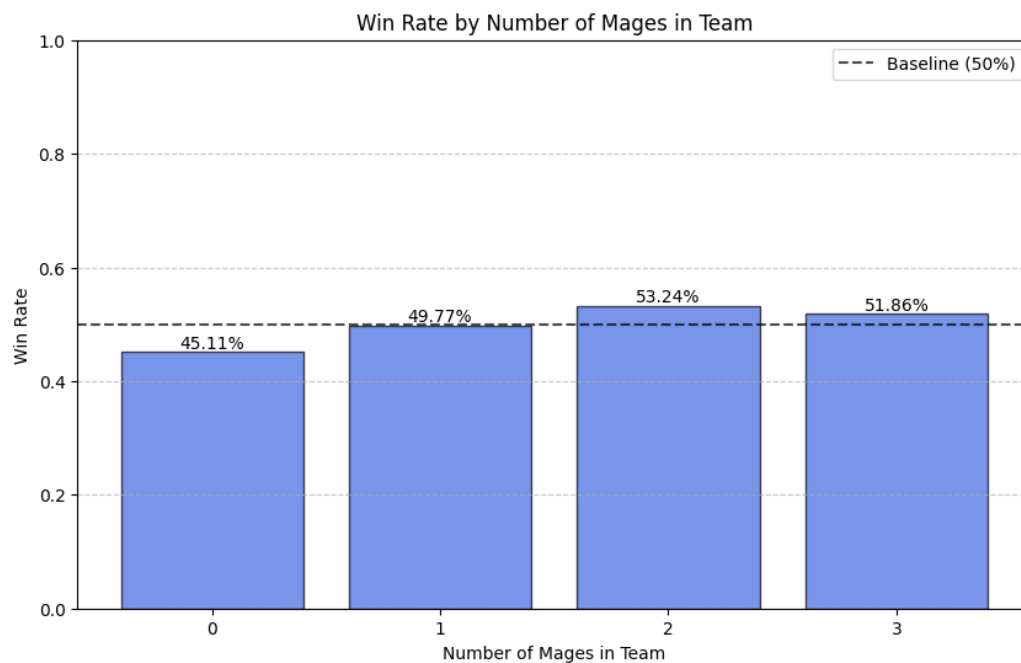


Figure 19: More Mage champions resulted in better win rate.

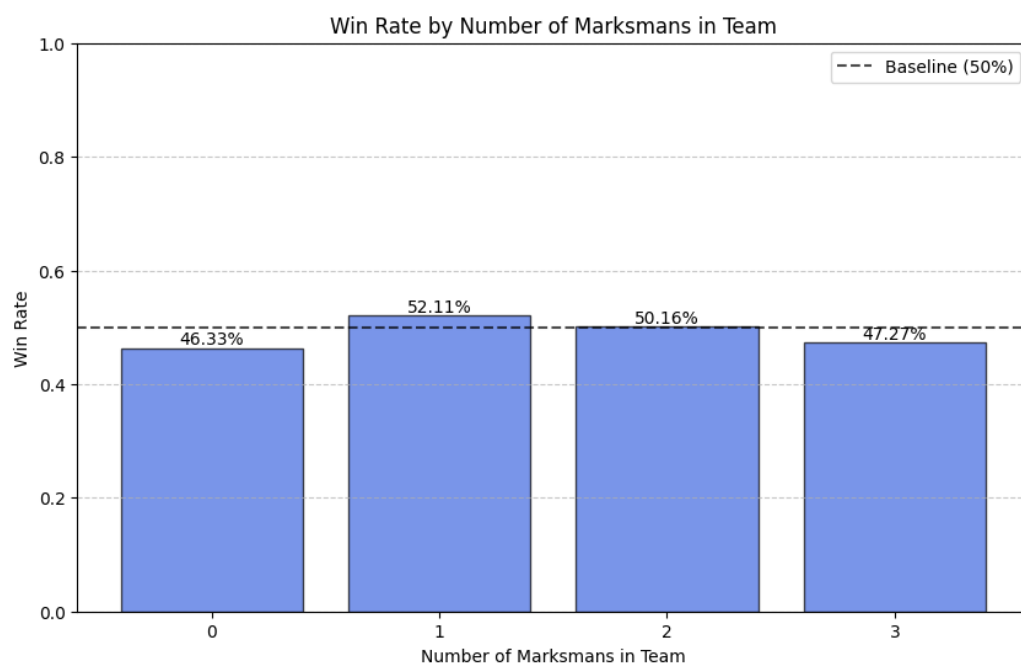


Figure 20: Having 1 Marksman on the team improves win rate.

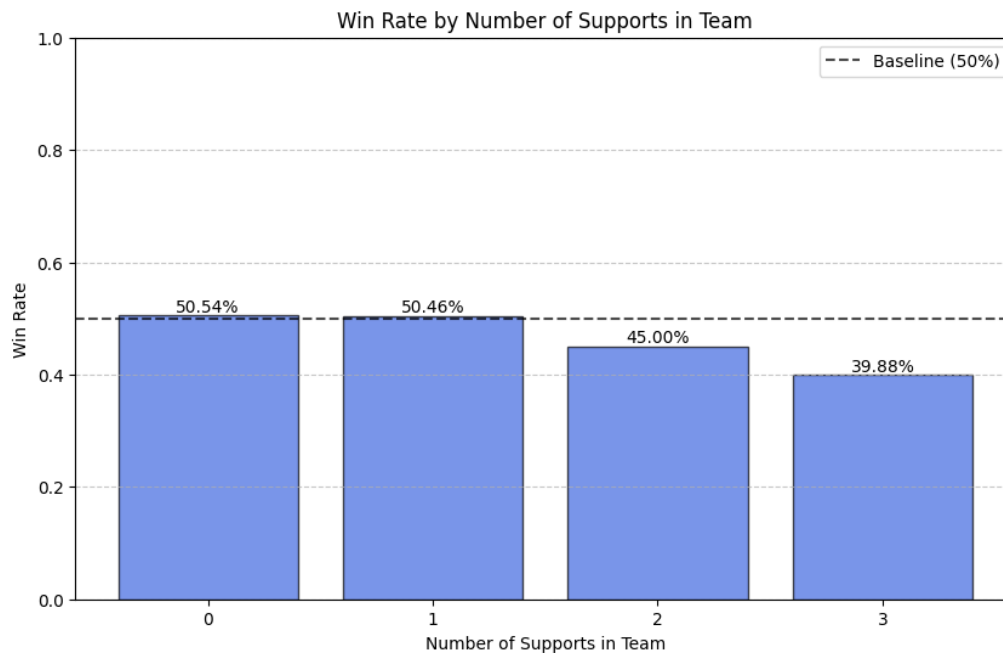


Figure 21: More than 1 Support decreases win rate.

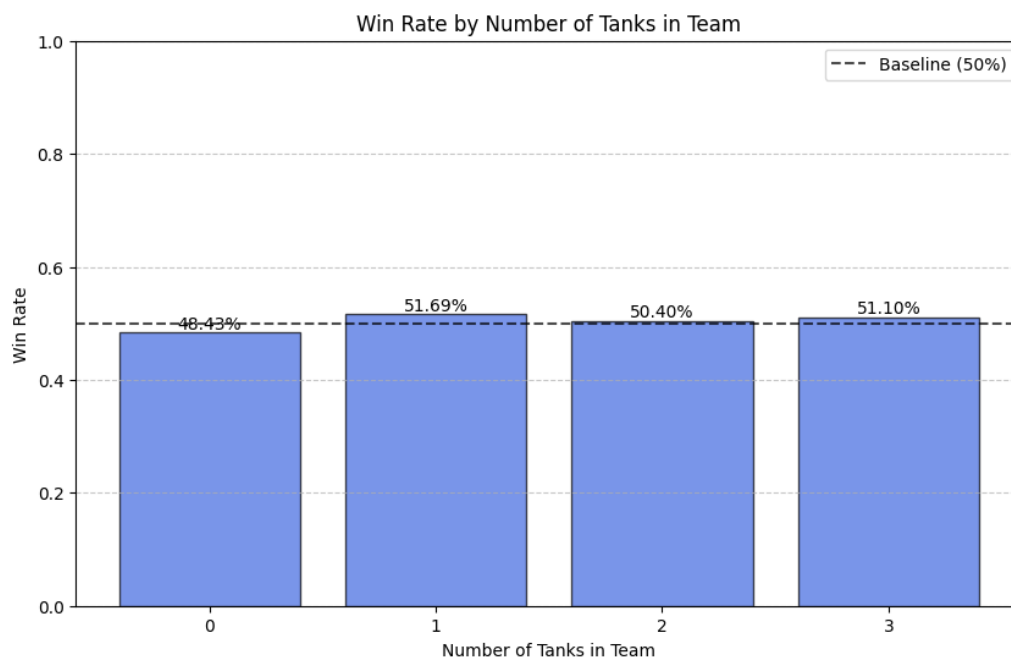


Figure 22: Having Tanks achieves >50% win rate, but does not change regardless of the amount.

The win rates show some patterns among the roles. Assassins have a clearly detrimental effect on win rate as their presence on teams increases dropping from 53% to 34%. Other roles show a peak in win rate by having 1 or 2 present but a decline on the extremes. Marksman win rates peak with 1 on the team, but decline if there are too many. Tank champion win rates appear stable for teams with at least 1 present.

Champion Statistics

Champion statistics also provide insight into team performance. The average ratings of *Attack*, *Defense*, *Magic*, and *Difficulty* can be examined across winning and losing teams to see if any key differences are apparent.

Comparison of Champion Statistics for Wins vs. Losses

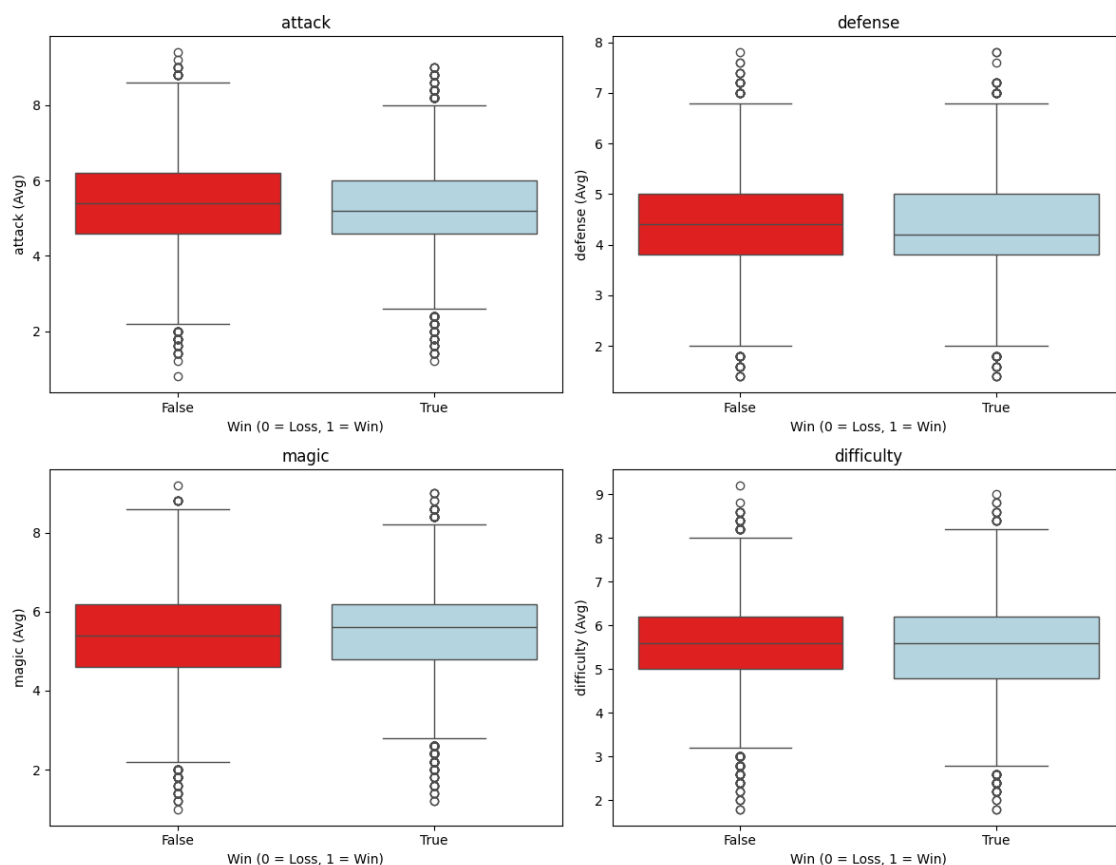


Figure 23: Distribution of champion statistics by match outcome.

Win	Avg. Attack	Avg. Defense	Avg. Magic	Avg. Difficulty
False	5.37	4.32	5.45	5.58
True	5.28	4.32	5.50	5.49
Difference	-0.09	0.00	+0.05	-0.09

Table 2: Average champion statistics and difference between winning and losing teams.

The distributions of champion statistics were similar between wins and losses, and the averages also did not vary much. Though the average *Magic* rating is slightly higher for winning teams, which aligns with the increasing win rate seen with the number of Mages on a team. The average *Attack* rating decreases for winning teams, also aligning with Marksman win rates decreasing beyond 1 Marksman.

Logistic Regression Model

Logistic regression was selected as the first predictive model due to its interpretability and ability to estimate the effect of each feature on binary match outcomes. Logistic regression is advantageous because it predicts the probability of a team winning a match. This makes it a suitable choice for the binary classification problem, where the target variable is win (1) or loss (0). This type of model also determines the effect of multiple predictor variables, the strength of which is represented through the feature's coefficient. The coefficient also provides the direction of the effect, showing how the probability of winning changes as that feature changes (Yasar et al., 2024).

Some disadvantages to logistic regression are its difficulty in modeling non-linear interactions. The role win rates showed that ARAM team compositions may have more complex interactions that may not fully be captured by a logistic regression model. Additionally, the model is sensitive to multicollinearity, where correlations between predictors can diminish the ability for the model to evaluate individual variables (Yasar et al., 2024).

Base Model

An initial logistic regression model was trained using all champion statistics and role count features to establish baseline model performance metrics. The variables were scaled using the *StandardScaler* provided in the *scikit-learn* package. The data was then split into training and test sets using an 80 / 20 split.

```
Baseline Logistic Regression Results

Features Included:
Index(['attack_mean', 'defense_mean', 'magic_mean', 'difficulty_mean',
      'primary_role_Assassin_sum', 'primary_role_Fighter_sum',
      'primary_role_Mage_sum', 'primary_role_Marksman_sum',
      'primary_role_Support_sum', 'primary_role_Tank_sum', 'win'],
      dtype='object')

Accuracy: 0.5431
Precision: 0.5394
Recall: 0.5998
AUC-ROC: 0.5598

Confusion Matrix:
[[1669 1763]
 [1378 2065]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.55	0.49	0.52	3432
1	0.54	0.60	0.57	3443
accuracy			0.54	6875
macro avg	0.54	0.54	0.54	6875
weighted avg	0.54	0.54	0.54	6875

```

Log-Likelihood: -4723.606509299363
Null Log-Likelihood: -4765.378066345869
McFadden's R-squared: 0.008765633379963278
Durbin-Watson statistic: 1.9917223814071952

```

Figure 24: Initial baseline logistic regression model results.

The baseline model resulted in an overall accuracy of 54.31%, exceeding the 50% of random chance but not by a significant amount. Recall reached 60%, indicating the model excelled at identifying actual positive cases, but did worse at reducing false positives with a precision of 53.94%. The log-likelihood of the model was -4723.61 and provides a baseline to compare to other version of the logistic regression model. The Durbin-Watson statistic was 1.99, with 2.0 being desirable, indicating that the independence of errors assumption holds for this model (Yasar et al., 2024).

Variance Inflation Factor

Since logistic regression is sensitive to highly correlated predictors, Variance Inflation Factor (VIF) values were calculated for all features. A high VIF (>10) indicates that a feature is highly collinear with others and may need to be removed. VIF values for the initial model predictors indicate the presence of correlations, especially with the Fighter, Marksman, and Mage role counts which had VIF values of 14.09, 16.85, and 17.95 respectively.

Feature	VIF
primary_role_Mage_sum	17.951917
primary_role_Marksman_sum	16.853047
primary_role_Fighter_sum	14.096807
primary_role_Tank_sum	9.925594
primary_role_Support_sum	5.246082
primary_role_Assassin_sum	5.192256
attack_mean	2.463966
magic_mean	2.136964
defense_mean	1.816627
difficulty_mean	1.275114

Figure 25: VIF values for all predictor variables showing significant correlation between predictors.

Instead of just removing the high VIF predictors, the correlation matrix may provide details on which predictors are correlated together to make more informed choices on which to remove. The matrix in Figure 26 shows that Mage counts are positively correlated with *magic_mean* and negatively correlated with *attack_mean*. Marksman count correlations show the reverse pattern. This is expected as Mages specialize in magic attacks, and Marksmen are high in physical attacks and damage.

The *attack_mean* and *magic_mean* columns were removed since these patterns are already captured by the presence of Mage and Marksman champion roles. This removal also meant sacrificing finer details about champion capabilities. However, role counts were prioritized for interpretability, ensuring players could more easily relate the model's insights to in-game decision-making. The resulting VIF values (Figure 27) are much reduced, with all predictors scoring under 10.

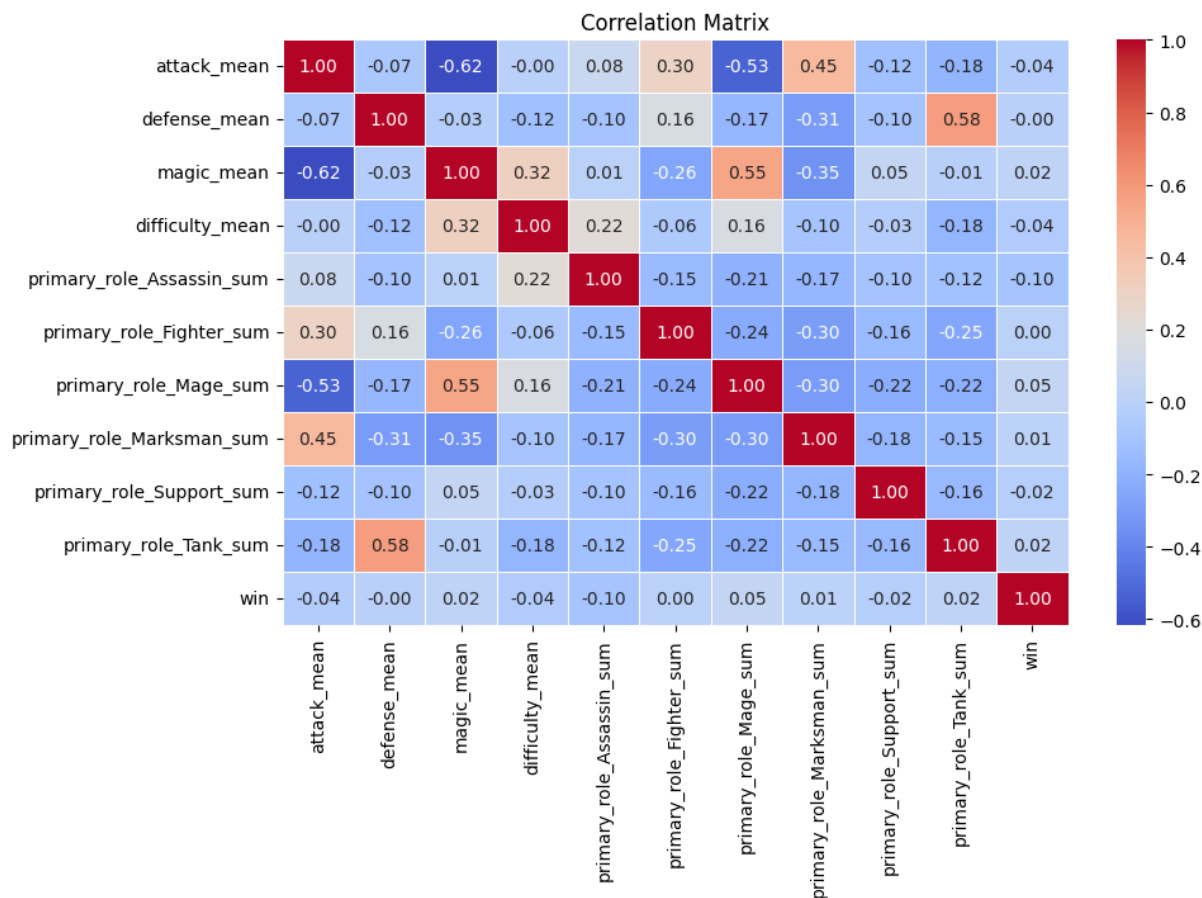


Figure 26: Correlation between initial model predictor variables.

Feature	VIF
primary_role_Mage_sum	9.249848
primary_role_Tank_sum	7.554222
primary_role_Fighter_sum	7.496036
primary_role_Marksman_sum	6.312183
primary_role_Assassin_sum	3.198446
primary_role_Support_sum	3.175989
defense_mean	1.810891
difficulty_mean	1.117646

Figure 27: Predictor VIF values after attack_mean and magic_mean were removed.

The model was re-run with the reduced number of predictor variables. The result, shown in Figure 28, was largely unchanged. Accuracy scored 54.01%, down from 54.31%. Recall and precision remained the same at 60% and 54%. The log-likelihood dropped to -4726.4, indicating the baseline model generally performed better. The Durbin-Watson statistic dropped to 1.98 but is still close to 2.0 to confirm independence of errors.

```

VIF-Reduced Logistic Regression Results

Features Included:
Index(['defense_mean', 'difficulty_mean', 'primary_role_Assassin_sum',
      'primary_role_Fighter_sum', 'primary_role_Mage_sum',
      'primary_role_Marksman_sum', 'primary_role_Support_sum',
      'primary_role_Tank_sum'],
      dtype='object')

Accuracy: 0.5401
Precision: 0.5364
Recall: 0.6018
AUC-ROC: 0.5571

Confusion Matrix:
[[1641 1791]
 [1371 2072]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.54	0.48	0.51	3432
1	0.54	0.60	0.57	3443
accuracy			0.54	6875
macro avg	0.54	0.54	0.54	6875
weighted avg	0.54	0.54	0.54	6875

```

Log-Likelihood: -4726.3982224952215
Null Log-Likelihood: -4765.378066345869
McFadden's R-squared: 0.008179800911481072
Durbin-Watson statistic: 1.984819734345351

```

Figure 28: Model results for the VIF-reduced model with *attack_mean* and *magic_mean* predictors removed.

Recursive Feature Elimination

The model can be further optimized using recursive feature elimination (RFE) to automatically select significant features while maintaining model accuracy. This method works by creating an initial model, then removing the least important features until an optimal subset of features remains (Magedmallek, 2024). The RFE method retained 4 out of the 8 original features of the VIF-reduced model. Specifically, *difficulty_mean*, and role counts for Assassin, Mage, and Tank champions were retained. The 4 retained features were input into a final version of the model, with results shown in figure 30.

```

Selected features: Index(['difficulty_mean', 'primary_role_Assassin_sum', 'primary_role_Mage_sum',
      'primary_role_Tank_sum'],
      dtype='object')

Removed features: Index(['defense_mean', 'primary_role_Fighter_sum', 'primary_role_Marksman_sum',
      'primary_role_Support_sum'],
      dtype='object')

Optimal number of features: 4 out of 8

```

Figure 29: RFE results showing the 4 selected features for final model.

```

RFE-Reduced Logistic Regression Results

Features Included:
Index(['difficulty_mean', 'primary_role_Assassin_sum', 'primary_role_Mage_sum',
      'primary_role_Tank_sum'],
      dtype='object')

Accuracy: 0.5455
Precision: 0.5399
Recall: 0.6242
AUC-ROC: 0.5541

Confusion Matrix:
[[1601 1831]
 [1294 2149]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.55	0.47	0.51	3432
1	0.54	0.62	0.58	3443
accuracy			0.55	6875
macro avg	0.55	0.55	0.54	6875
weighted avg	0.55	0.55	0.54	6875

```

Log-Likelihood: -4729.461277084867
Null Log-Likelihood: -4765.378066345869
McFadden's R-squared: 0.0075370282821114465
Durbin-Watson statistic: 1.96992


```

	Feature	Coefficient	Change in Odds
0	difficulty_mean	-0.067559	0.934672
1	primary_role_Assassin_sum	-0.153414	0.857775
2	primary_role_Mage_sum	0.099063	1.104136
3	primary_role_Tank_sum	0.047809	1.048970

Figure 30: Final logistic regression model after RFE reduction.

The final model included only the 4 predictor variables retained by the RFE method. It achieved an accuracy of 54.55%, barely improved from the 54.01% of the previous version, but still slightly above random guessing. The log-likelihood value of -4729.46, shows a small but measurable improvement in model fit compared to the null log-likelihood of -4765.37. The low McFadden's R^2 (0.00757) indicates the model explains less than 1% of the variance in match outcomes. This suggests that while the features included contribute to predicting match results, other external factors likely play a larger role.

Model Interpretation

The coefficients of the predictor variables can be used to construct an equation to estimate the log-odds of winning given a certain team composition. The sign and magnitude of the coefficients indicate the strength and direction of the effect each has on the log-odds of the outcome. The final model can be expressed in the equation:

$$\text{logit}(\text{win}) = 0.002 - (0.07 * \text{difficulty_mean}) - (0.15 * \text{primary_role_Assassin_sum}) + (0.10 * \text{primary_role_Mage_sum}) + (0.05 * \text{primary_role_Tank_sum})$$

The exponential of the coefficients represents the change in log-odds for each unit of change in that predictor variable. Because the variables were scaled though, the unit of change is measured as a change in the standard deviation rather than raw data. For example, an increase in 1 standard deviation of number of Assassin champions on a team results in a 14% decrease in the chance of winning (odds change of 0.86). An increase in one standard deviation of Mage champions on a team results in an increase of 10% chance of winning (odds change of 1.10).

Both average champion difficulty rating and count of Assassin champions have a detrimental effect on win probability. This matches the win rate trend seen with increasing number of Assassin champions on a team. On the other end, higher Mage and Tank champion counts improve win probability as expected. Defense ratings, Fighter, Marksman, and Support roles were not included in the final model. This may present a challenging interpretation to players if the only other champions available after a re-roll are not Mage or Tank types.

The final equation can be programmed into a function to return win probabilities based on the champion names inputted. Figure 31 shows a custom function that takes a list of champion names, references their roles and statistics, and uses the logistic regression equation to produce a probability of winning the ARAM match.

```
def predict_ARAM_win_chance(champ_names):
    team_data = champ_info[champ_info['name'].isin(champ_names)]

    # Compute team features
    difficulty_mean = team_data['difficulty'].mean()
    role_sums = team_data.loc[:, ['primary_role_Mage', 'primary_role_Assassin',
                                'primary_role_Tank']].sum()

    # Create feature array and scale
    team_features = np.array([difficulty_mean] + role_sums.tolist()).reshape(1, -1)
    scaled_team_features = scaler.transform(team_features)

    # Coefficients from trained logistic regression model
    coefficients = np.array([-0.0676, -0.1534, 0.0990, 0.0478])

    # Compute log-odds and convert to win probability
    log_odds = np.dot(scaled_team_features, coefficients)
    win_probability = expit(log_odds)[0]

    return print(f"""\nTeam Champions: {' '.join(champion_team)}
Predicted Win Probability: {win_probability:.4f}\n""")
```

Figure 31: Custom function to calculate probability of winning an ARAM match based on the champions inputted.

Inputting an example team consisting of the champions: Ashe (Marksman), Veigar (Mage), Garen (Tank), Akali (Assassin), and Anivia (Mage), the model predicts a 45.26% probability of winning an ARAM match. Modifying the team to a Tank and Mage heavy team of: Galio (Tank), Cho'Gath (Tank), Morgana (Mage), Ziggs (Mage), and Fiddlesticks (Mage) results in a much higher win probability of 60.94%.

Random Forest Model

Random forest is an ensemble learning method that builds multiple decision trees and combines their predictions to improve classification accuracy. Unlike logistic regression, which assumes a linear relationship between predictors and the outcome, random forest can capture complex, non-linear interactions between variables. Each tree in the forest is trained on a randomly selected subset of the data, and the final prediction is determined by aggregating the individual tree outputs. For a classification problem such as predicting probability of win, this is the average of the individual tree outputs (IBM, 2024).

One of the key advantages of using random forest for this project is its ability to model non-linear interactions between team composition features. In ARAM matches, benefits from champions of a certain role may not be linear. For example, having 1 or 2 Marksman champions might be beneficial, but there are diminishing returns beyond that amount. This flexibility allows random forest to detect interactions that logistic regression may struggle to capture (IBM, 2024).

However, a drawback of random forest is its limited interpretability compared to logistic regression. While logistic regression provides clear coefficients that indicate the direction and magnitude of each feature's effect, random forest produces a sequence of feature values (a branch) where the outcome is determined by following that sequence rather than a general rule. This makes it more challenging to extract direct insights.

Baseline Model

To establish an initial benchmark, a baseline random forest model was trained using all available predictor variables, including champion statistics and role counts. The model was set up with 500 trees and a minimum leaf size of 50 to prevent overfitting and extreme effects from rare team combinations. The dataset was split using an 80/20 train-test split ratio, consistent with the logistic regression model. The baseline model achieved an accuracy of 55.03% and an AUC-ROC score of 0.57, indicating its ability to distinguish between winning and losing teams slightly better than random chance.

```

Random Forest Base Model Accuracy: 0.5503
Random Forest AUC-ROC: 0.5684

Confusion Matrix:
[[1812 1620]
 [1472 1971]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.55	0.53	0.54	3432
1	0.55	0.57	0.56	3443
accuracy			0.55	6875
macro avg	0.55	0.55	0.55	6875
weighted avg	0.55	0.55	0.55	6875

Figure 32: Baseline random forest model results.

Feature Importance

One of the benefits of random forest is that it provides importance scores, which identify which predictors contribute the most to the model's decisions. Feature importance values were calculated using the Gini impurity reduction method, which measures how much each variable contributes to reducing classification uncertainty (IBM, 2024).

The base model feature importance rankings (Figure 33) revealed that the average attack, difficulty, and magic ratings for a team were the most influential predictors, while champion roles, specifically Tank, Fighter, and Support, were the least important. Despite their predictive strength, these champion statistics were removed in favor of role counts to improve model interpretability. While a decision tree split like "*difficulty_mean* should be between 2.5 and 6.5" may optimize accuracy, it does not translate into actionable decisions for players. In an ARAM match lobby, players cannot realistically calculate the average difficulty of their team's champions before deciding whether to re-roll.

By contrast, role counts such as "0 Assassins, 1-2 Mages, and at least 1 Tank" are immediately recognizable and actionable. Players can make informed decisions immediately, without requiring additional calculations. Therefore, despite a slight tradeoff in model accuracy, champion statistics were excluded in favor of a simplified, role-based model that aligns better with real-world decision-making in ARAM.

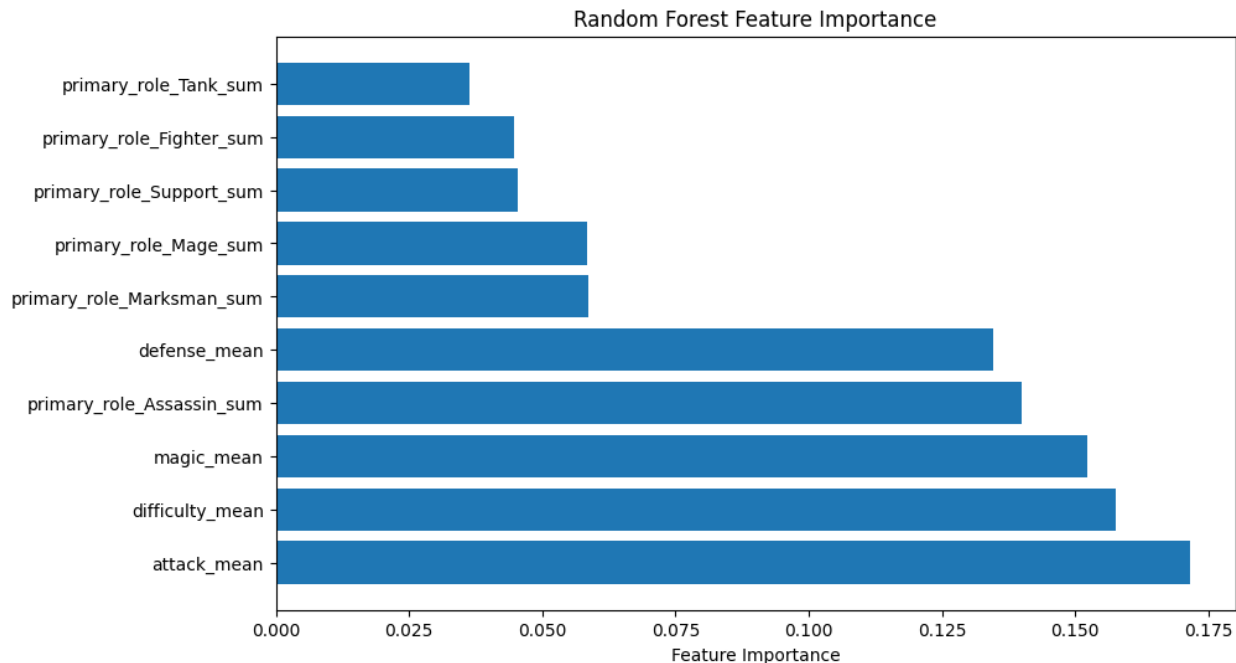


Figure 33: Feature importance for baseline random forest model.

Final Model

After removing champion statistics and retaining only role counts, the Random Forest model was re-trained using the reduced feature set. The goal of this refinement was to balance predictive performance with practical interpretability, ensuring that the model's outputs could provide meaningful insights that players could act upon in real-time.

Despite the removal of high-importance features like *difficulty_mean* and *attack_mean*, the optimized model maintained a similar performance, achieving an accuracy of 54.89% and an unchanged AUC-ROC score of 0.57. This suggests that the predictive strength of champion statistics was largely captured by the role counts.

The feature importance scores after the refinement showed that in the absence of champion statistics, Assassin, Mage, and Marksman counts became the new dominant predictors, reinforcing earlier findings that team composition, especially regarding the presence of Assassins and Mage champions is a key driver of match success in ARAM.


```

Random Forest Base Model Accuracy: 0.5489
Random Forest AUC-ROC: 0.5676

Confusion Matrix:
[[1760 1672]
 [1429 2014]]

Classification Report:
              precision    recall  f1-score   support

     0       0.55       0.51       0.53       3432
     1       0.55       0.58       0.57       3443

 accuracy      0.55
 macro avg     0.55
weighted avg     0.55

```

Figure 34: Reduced random forest model results containing only champion role counts.

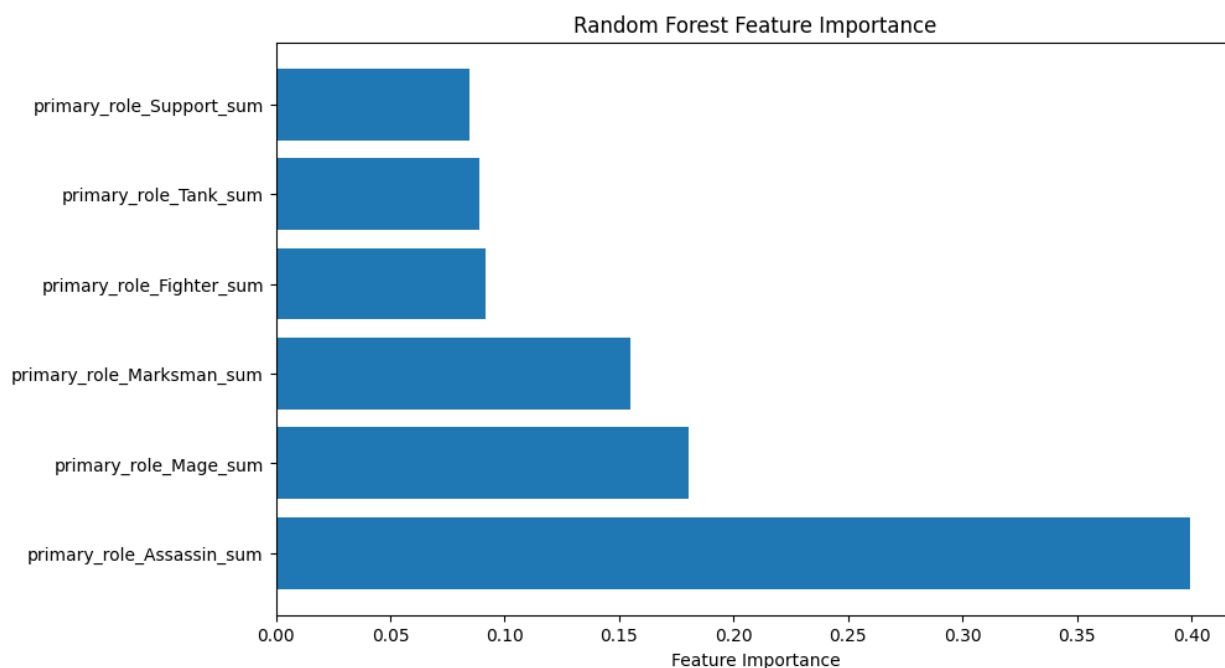


Figure 35: Feature importance in reduced random forest model using only champion role counts.

Model Interpretation

Although Random Forest consists of multiple trees working together, extracting and analyzing a single decision tree from the model provides insights into how specific team compositions influence match outcomes.

A sample tree from the final model (Figure 36) illustrates decision splits based on role distributions, making it possible to interpret how different team compositions impact win probability. It also demonstrates how non-linear interactions were captured, showing the optimal number for each role rather than a general direction of effect.

The tree shows that:

- Teams with 0 Assassins, 2 Mages, 2 Tanks, and 1 Marksman have highest win probability at 61.5%.
- Having any Assassins, especially more than 1, dramatically reduces win probability.
- Teams with no Marksmen also struggle to achieve high win probabilities.

By structuring the model around role counts rather than raw champion statistics, these insights are immediately actionable for players in an ARAM match. Instead of needing to calculate a team's average difficulty or attack rating, they can quickly assess whether their role distribution aligns with winning trends.

```
Highest Win Probability: 0.615 (Leaf 87)
Lowest Win Probability: 0.351 (Leaf 119)

Decision Path for Highest Win Probability:
primary_role_Assassin_sum <= 0.50 → primary_role_Mage_sum > 1.50 →
primary_role_Marksman_sum > 0.50 → primary_role_Tank_sum > 0.50 →
primary_role_Mage_sum <= 2.50 → primary_role_Marksman_sum <= 1.50 →
primary_role_Tank_sum > 1.50

Decision Path for Lowest Win Probability:
primary_role_Assassin_sum > 0.50 → primary_role_Mage_sum > 0.50 →
primary_role_Marksman_sum <= 0.50 → primary_role_Fighter_sum > 0.50 →
primary_role_Assassin_sum > 1.50
```

Figure 36: Example decision tree showing path for high and low win probabilities based on role counts.

To demonstrate how the final model can be used in practice, a function was developed to predict match outcomes based on a given set of champion names. This function references each champion's role, aggregates the team's role distribution, and applies the trained Random Forest model to calculate a win probability.

```
def predict_ARAM_win_chance_rf(champ_names):
    role_counts = {
        "primary_role_Assassin_sum": 0,
        "primary_role_Fighter_sum": 0,
        "primary_role_Mage_sum": 0,
        "primary_role_Marksman_sum": 0,
        "primary_role_Support_sum": 0,
        "primary_role_Tank_sum": 0
    }

    # Map champion names to roles
    for champ in champ_names:
        roles = champ_info.loc[champ_info["name"] == champ, [
            "primary_role_Assassin", "primary_role_Fighter", "primary_role_Mage",
            "primary_role_Marksman", "primary_role_Support", "primary_role_Tank"
        ]]
        role_counts = {key: role_counts[key] + roles.iloc[0][key.replace("_sum", "")] for key in role_counts}

    input_df = pd.DataFrame([role_counts])
    win_prob = rf_base.predict_proba(input_df)[0, 1][0]

    return print(f"""\nTeam Champions: {' '.join(champ_names)}
Predicted Win Probability: {win_prob:.2%}\n""")
```

Figure 37: Custom function to predict win probability using champion names and random forest model.

Using the same example teams as the logistic regression example, the first team with: Ashe (Marksman), Veigar (Mage), Garen (Tank), Akali (Assassin), Anivia (Mage) achieves a win probability of 50.00%. The second team with Galio (Tank), Cho'Gath (Tank), Morgana (Mage), Ziggs (Mage), Fiddlesticks (Mage) achieves a win probability of 53.25%. The second team, featuring more Tanks and Mages, yields a higher win probability, consistent with the example decision tree splits. These predictions reinforce the model's insights that role balance, particularly the presence of Assassins and Mages, plays a crucial role in team success.

Data Summary and Implications

The objective of this study was to determine whether team composition in ARAM (All Random All Mid) matches influences match outcomes and if a predictive model could use team composition to predict match outcomes with an accuracy greater than 50%, outperforming random guessing.

The results indicate that certain role compositions consistently perform better, supporting the notion that despite the random champion selection, ARAM match outcomes can be optimized using re-rolls in pre-match decision making. Both the logistic regression and random forest models achieved accuracies slightly above 50%, allowing for rejection of the null hypothesis and confirming that a model can predict ARAM match outcome better than random chance. The logistic regression model achieved a final accuracy of 54.55% while the random forest model achieved a final accuracy of 54.89%.

These results confirm predictability beyond pure chance. However, relatively modest accuracy suggests other external factors not considered in this analysis, such as player skill, item purchases, and in-game decisions even more significantly impact match outcomes.

Model Comparison

Two different modeling approaches were used to predict ARAM match outcomes: logistic regression and random forest. While both models provided moderate accuracy improvements over random guessing, they differed in their interpretability, ability to capture complex interactions, and practical application for players.

Predictive Performance

Both models achieved similar accuracy, with logistic regression reaching 54.55% and random forest achieving 54.89%. While random forest performed slightly better, the difference was marginal, suggesting that the overall effect of team composition on match

outcomes is present but not overwhelmingly strong. This suggests that while champion roles contribute to a team's chances of winning, external factors still play a dominant role.

The random forest model produced a higher AUC-ROC score (0.57 compared to logistic regression's 0.55), indicating that it was slightly better at distinguishing between winning and losing teams. This suggests that the non-linear interactions between roles, which logistic regression cannot model effectively, provide additional predictive value.

Interpretability

One of the major advantages of logistic regression is its clear feature coefficients, which allow for straightforward interpretation. In this model, each role count had a direct coefficient, making it easy to quantify its effect on win probability. For example, the model clearly indicated that each additional Assassin champion lowered win probability, while the presence of Mages and Tanks increased it. This makes logistic regression useful for players who want simple, direct guidelines on how each role impacts match outcomes.

By contrast, random forest lacks direct interpretability but provides valuable insights through its decision trees. Rather than assigning fixed weights to roles, the model evaluates team compositions holistically, considering role interactions and threshold effects. For instance, the model identified that 1-2 Mages improved win probability, but adding more provided diminishing returns. This type of insight is difficult to extract from logistic regression, where each role is treated independently rather than in combination with others. Additionally, feature importance rankings in the random forest model revealed that Assassin, Mage, and Marksman counts were the strongest predictors once champion statistics were removed. This aligns with player intuition but also highlights the importance of team composition over raw champion stats, reinforcing the role-based approach taken in this study.

Practical Application

For real-world applications, each model presents unique advantages. Logistic regression is better suited for players looking for quick, easily digestible general insights. A player can glance at the model results and understand, for example, that stacking Assassins tends to reduce win probability while increasing Mages is likely beneficial.

On the other hand, random forest provides a more nuanced view of composition balance, making it better suited for strategic decision-making where players can adjust the exact team composition dynamically. Rather than relying on general directions (more Mages), random forest can be used to optimize the exact number of each role on a team to produce the highest probability of winning (0 Assassins, 2 Mages, 2 Tanks, 1 Marksman).

Ultimately, if a player wants a simple rule-based approach, logistic regression is more useful. But if they are looking for a more flexible tool to explore specific team compositions, random forest provides a deeper level of insight.

Analysis Limitations

The models in this study successfully demonstrated that team composition plays a role in ARAM match outcomes, but several limitations must be acknowledged. These limitations primarily stem from missing features and the limited scope that results from considering only pre-game team composition.

One of the most significant limitations of this analysis is that team composition alone does not fully determine match outcomes. Roles like Mages, Tanks, and Assassins showed measurable effects on win probability, but other key factors were not included in the models such as:

- **Player Skill:** Even with an optimal composition, teams can struggle to win if one or more players are not adept at playing their selected champion, giving the enemy team a skill advantage regardless of their composition
- **Champion Synergies:** The models do not explicitly analyze champion synergies such as having a Support + Marksman role combination which may be more beneficial to have than simply more Marksmen.
- **In-Game Adaptations:** The goal of the models was to only consider pre-match composition of a single team. During the match though, item builds, ability upgrades, and team communication can significantly influence results. These in-game features likely drive the large portion of variance in match outcome beyond team composition.
- **Opposing Team Composition:** The data points used only consider one side of the match since the enemy team is not known until champions selections are locked in. However, the composition of the enemy team also matters and models may perform better when considering both teams together as opposed to one team in isolation.

This study was purposefully limited to the scope of a single team's pre-match champion selections. The limitations above suggest that future research should incorporate additional features, including dynamic in-game match conditions, to improve prediction accuracy. Expanding the models to include such features could provide insights that can be actioned on both during pre-match champion selection and during the match to assist teams in optimizing probability of winning despite ARAM's inherent randomness.

Recommended Actions

The findings of this study confirm that team composition in ARAM has a measurable effect on match outcomes, even though randomness and in-game factors still play a significant role. Based on the model insights, several actionable strategies can help players optimize their champion selection decisions, particularly when deciding whether to re-roll, trade champions, or adjust playstyle based on team composition.

Applying Model Rules

One of the strongest takeaways from both models is that a balanced but strong team composition improves win probability. Teams that include a mix of durable frontliners (Tanks), and sustained damage dealers (Mages, Marksmen) tend to perform better than teams with high role redundancies. Players should apply the general rules from the models in champion selection, with the following specific recommendations:

- **Ensure at least one Tank** – Both models show that at least one Tank champion improves the probability of winning, holding all else equal.
- **Avoid any Assassins** – Having Assassin champions, even 1, had a clear detrimental effect on win probability and should be completely avoided.
- **Prioritize Mages** – The presence of 1 or 2 Mages consistently improved win probability. If the rest of the team looks well balanced, players should prioritize selecting a Mage champion over other roles.

Since re-rolls are limited to 2 per match and take time to replenish between matches, players should only use a re-roll to achieve the above recommendations. Re-rolling simply because a champion is unfamiliar or disliked should be avoided if the re-roll results in violating the recommendations.

Beyond these main recommendations, players should opt for a balanced team, using champions they are comfortable playing. Teams should communicate during champion selection and coordinate trades so each player has a champion they are comfortable with even if the role composition doesn't change.

ARAM Team Building Tool

Another recommendation would be the development of a tool that uses the resulting models to provide live recommendations in champion selection. This could take the form of a simple tool where players input their champions, and it outputs win probability predictions based on the role distribution. Or a re-roll guidance system that suggests whether to re-roll based on current team composition and the chances of the

next champion improving win probability. Such a tool would help players make data-driven decisions in real-time and further optimize their ARAM match outcomes.

Ultimately, while no model can guarantee a win, understanding how roles influence match outcomes and following these recommended actions in applying the model results gives players an advantage in navigating the chaos and randomness of ARAM.

Future Research Direction

While this study provides valuable insights into how team composition affects ARAM match outcomes, there are several opportunities for further exploration. Expanding on this research could lead to more accurate predictive models, deeper understanding of gameplay mechanics, and practical tools for players. Two promising directions for future study include enhancing model features with player-specific data and exploring alternative machine learning techniques to improve predictive accuracy.

Incorporating Player Skill and In-Game Factors

The biggest limitation of this study is that it treats all players as equal, assuming team composition alone determines match outcomes. However, player skill and in-game decisions likely play a more significant role in ARAM success. Future research could improve predictive models by incorporating player-specific data, including:

- **Player Specific ARAM Skill:** A player's skill level in LoL might significantly impact their team's performance. Teams with higher-ranked players may have an inherent advantage, even if their team composition is suboptimal.
- **Champion Mastery Levels:** Players who frequently play a specific champion may perform better with that champion, even if it's not an optimal pick for their team.
- **Itemization and Adaptation Strategies:** Tracking item choices and playstyle could reveal insights into how teams can overcome composition weaknesses.

By integrating player-based factors, future models could better distinguish whether a team won because of their composition or because they simply had more skilled players or better in-game items and playstyle. This would improve the model's accuracy and provide more personalized insights for players looking to optimize their ARAM performance.

Explore Other Machine Learning Techniques

The models used in this study, logistic regression and random forest, were chosen for their interpretability and structured decision-making capabilities. However, modifications to these models or other advanced machine learning techniques could potentially yield higher accuracy by capturing hidden relationships within the data.

Future research could experiment with more complex interaction terms between champion roles, or transformed terms for such as squared or square-root terms for single roles, modeling non-linear patterns in for logistic regression.

Neural Networks & Deep Learning: ARAM match outcomes may be influenced by complex, multi-factor interactions that linear models struggle to capture. A deep learning approach, such as a feedforward neural network or transformer-based model, could detect more nuanced team synergy effects.

While these techniques might provide better predictive accuracy, they also come with trade-offs, primarily in interpretability and usability for players. Future research should explore how to balance accuracy with practical application, ensuring that models remain useful for decision-making without becoming overly complex and losing applicability.

By incorporating player skill data and in-game factors, as well as testing alternative machine learning techniques, future research can enhance predictive modeling for ARAM matches. These advancements could lead to more precise win probability estimates, better strategic insights for players, and potentially even AI-driven coaching tools. Future research informed by these directions has the potential to push the boundaries of esports analytics even further, uncovering deeper insights into how strategy, skill, and randomness interact in ARAM and competitive gaming as a whole.

References

- Clement, J. (2025, January 16). *Esports market - statistics & facts | statista*. Statista.
<https://www.statista.com/topics/3121/esports-market/>
- Gill, S. (2025, January 2). *League of Legends Player Count & Stats 2025*. Priori Data.
<https://prioridata.com/data/league-of-legends/>
- Riot Games. (2024). *League of Legends How to play*. League of Legends How to Play.
<https://www.leagueoflegends.com/en-us/how-to-play/>
- ARAM. League of Legends Wiki. (2025, February 13).
<https://wiki.leagueoflegends.com/en-us/ARAM>
- LoL Tournaments. Duoo. (2023). <https://duoo.gg/tournaments>
- Riot Games. (n.d.). *Developer API Policy*. Riot Developer Portal.
https://developer.riotgames.com/docs/lol#data-dragon_champions
- Item. League of Legends Wiki. (2025b, January 29).
<https://wiki.leagueoflegends.com/en-us/Item>
- List of champions/base statistics. League of Legends Wiki. (2024, March 2).
https://wiki.leagueoflegends.com/en-us/List_of_champions/Base_statistics
- Chaudhary, S. (2024, January 24). *1.5 IQR rule explained*. Built In.
<https://builtin.com/articles/1-5-iqr-rule>
- Yasar, K., Lawton, G., & Burns, E. (2024, April 25). *What is logistic regression?: Definition from TechTarget*. Search Business Analytics.
<https://www.techtarget.com/searchbusinessanalytics/definition/logistic-regression>
- Magedmallek. (2024, June 18). *Feature selection explained: RFECV*. Kaggle.
<https://www.kaggle.com/code/magedmallek/feature-selection-explained-rfecv>
- IBM. (2024, December 19). *What is Random Forest?*. IBM.
<https://www.ibm.com/think/topics/random-forest>